

Missing return-value validation of the function PyArray_DescrNew #19038

✓ Closed
#20960

awen-li opened on May 19, 2021

Reproducing code example:

The definition of PyArray_DescrNew

```
NPY_NO_EXPORT PyArray_Descr * PyArray_DescrNew(PyArray_Descr *base)
{
    PyArray_Descr *newdescr = PyObject_New(PyArray_Descr, Py_TYPE(base));
    if (newdescr == NULL) {
        return NULL; -----> **point 1**
    }
    .....
    if (base->c_metadata != NULL) {
        newdescr->c_metadata = NPY_AUXDATA_CLONE(base->c_metadata);
        if (newdescr->c_metadata == NULL) {
            PyErr_NoMemory();
            /* TODO: This seems wrong, as the old fields get decref'd? */
            Py_DECREF(newdescr);
            return NULL; -----> **point 2**
        }
    }
    .....
    if (newdescr->subarray) {
        newdescr->subarray = PyArray_malloc(sizeof(PyArray_ArrayDescr));
        if (newdescr->subarray == NULL) {
            Py_DECREF(newdescr);
            return (PyArray_Descr *)PyErr_NoMemory(); -----> **point 3**
        }
    }
    .....
    return newdescr;
}
```

Call-site example for PyArray_DescrNew

```
NPY_NO_EXPORT PyArray_Descr *
PyArray_DescrNewByteorder(PyArray_Descr *self, char newendian)
{
    PyArray_Descr *new;
    char endian;
```

```
new = PyArray_DescrNew(self);
endian = new->byteorder; -----> direct read through "new"
.....
}
```

Error message:

At most call-sites for PyArray_DescrNew, there are no validations of its return, but an invalid address may be returned.

[example](#)

NumPy/Python version information:

the main branch

1

awen-li on Jun 1, 2021

Author ...

Anyone can help confirm this issue? thanks.

eric-wieser on Jun 1, 2021

Member ...

This looks valid to me, but probably only happens if memory is exhausted. A patch would be appreciated.

1

1

 **eric-wieser** added **00 - Bug** on Jun 1, 2021

00xc on Dec 21, 2021

...

Hi, any update on this issue? It was recently assigned [CVE-2021-41495](#).

mattip on Dec 21, 2021

Member ...

@00xc do you know how those CVEs are being generated? This is the second one in two days, and both of them seem rather far fetched. I would hate for NumPy to have to enter a sudden race to close issues in a mode of "CVE-driven development".

6

00xc on Dec 21, 2021

...

[@00xc](#) do you know how those CVEs are being generated? This is the second one in two days, and both of them seem rather far fetched. I would hate for NumPy to have to enter a sudden race to close issues in a mode of "CVE-driven development".

I have no idea who is requesting them - we get CVE reports from our automated fetchers. I do agree that this and [CVE-2021-41496](#) (which I assume is the other CVE you're referencing) seem to be far fetched, and not too security critical, but I just thought pinging upstream would not hurt.

2

mattip on Dec 21, 2021

Member ...

Thanks for the heads up.

legendlc on Jan 6, 2022 · edited by legendlc

Edits ...

Hi, any progress on this? Thanks.

The CVSS score of this vulnerability seems to outweigh its impact

Severity

CVSS Version 3.x

CVSS Version 2.0

CVSS 3.x Severity and Metrics:



NIST: NVD

Base Score: 7.5 HIGH

Vector:

CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

seberg on Jan 6, 2022

Member ...

This is only likely (or even possible) to cause issues when you are already out of memory, in that case you can a process crash, rather than a Python interpreter shutdown, I guess? ;)

These CVE reports (with such ridiculous scores for what I understand), are a serious nuisance by now. But I am not sure what we should do about it easily.

attritionorg on Jan 7, 2022

...

As a general FYI on the CVE and other vulnerability databases (VDB) process, one requirement of CVSS is "score for the worst". If a vulnerability is confirmed by the vendor (this was) and the impact is not entirely understood (like this), it will be scored for what the worst outcome might be. In this case, a full remote DoS which gives the CVSSv3 7.5 score.

As for what you can/should o? If there are limiting factors that make this harder to exploit (increasing access complexity) or the impact is not severe (partial availability), the vendor or researcher briefly explaining that is a huge help to outside parties that don't work with the software every day. Then the VDB analysts can more easily understand the issue and score accordingly.

2



But, a user cannot trigger this except by first exhausting memory (which was pointed out in the confirmation). Further, if someone can run arbitrary NumPy code they can already trivially DoS you (or worse), so the main security consideration that I see for NumPy at this point is malicious data. It is unclear to me that this fits the bill (or any/most of the other CVEs here).

I see multiple issues:

- This probably creates a lot of unnecessary work for downstream to review users who use NumPy well isolated from any malicious user.
- We were *not* asked for confirmation beyond on the issues here (which all seemed low priority for good reasons). We could have explained why they seemed not very important ~~nobody bothered too much about it.~~
- It is not a great dynamic if we end up having to scramble to fix meaningless CVEs.

1



attritionorg on Jan 7, 2022

...

"but probably only happens if memory is exhausted." That doesn't specify who can exhaust memory. It implies the attacker might be able to do that so "assume the worst" kicks in.

Your caveat of "if someone can run arbitrary NumPy code..", is *great* to point out. Those are the kinds of clarifications that are immensely helpful to third-party analysts not intimately familiar with the software.

re: "if this fits the bill". The general rule for a 'vulnerability' is if privilege boundaries can be crossed. Can a user introduce malicious code that lets them do something they should not be able to? In this case, create a memory exhaustion (limited DoS) scenario? Something else? Or is this all entirely moot because a user is expected to be trusted at the point they introduce that code? If that is the case, a vendor giving a clear dispute is what CVE needs. Once the ID is assigned, you can't un-ring that bell so to speak. But MITRE can flag an entry as "Vendor Disputed" and give the caveat, often quoted verbatim from the vendor. So a clear/concise one-liner explaining why it isn't a vulnerability is the best you can do if an erroneous report is filed (or the report is accurate, but not qualified as to the conditions for triggering).

The CVE ecosystem these days is not designed to get vendor confirmation. The original purpose of CVE was to assign a candidate ID (e.g. CAN-2022-0001) while it was discussed and determined to be valid (which happened behind the scenes of closed-source vendors typically). If not, it would be rejected and that CVE ID would never see the light of day. If valid, it would move to full ID and change from CAN to CVE. The software ecosystem changed, MITRE changed the CVE process, and the 'CAN' process was removed because it was effectively obsolete by ~ 2006.

These days just the presence of a CVE can be incredibly troublesome for vendors for many reasons, and add a lot of work that they should never have had to deal with. They almost invariably begin with an incomplete or inaccurate bug report that makes it seem worse than it is. I feel really bad for vendors that have to deal with this, but that is how CVE works unfortunately.

3



rgommers on Jan 7, 2022

Member

...

Your caveat of "if someone can run arbitrary NumPy code..", is *great* to point out. Those are the kinds of clarifications that are immensely helpful to third-party analysts not intimately familiar with the software.

...

Or is this all entirely moot because a user is expected to be trusted at the point they introduce that code?

This is true here, and pretty much always true for CVEs we get, that's what is so frustrating. It's a Python library for numerical analysis, that for 99.9x% of usage *is not exposed to untrusted users*. The only users that should really worry about this are things like services that provide hosted runners (Binder, Colab, etc.). And those allow executing arbitrary Python code, and hence they are very much aware that they should sandbox and secure their runners.

The general rule for a 'vulnerability' is if privilege boundaries can be crossed.

You could argue that there's things like HPC systems which are shared between users, and those could use a vulnerability to cross a boundary. Which is technically true, but those users are typically already trusted (e.g. employees or students within an organization), which is hugely different from CVEs related to, say, components from which websites are built. So yes, it's indeed largely moot.

.... I feel really bad for vendors that have to deal with this, but that is how CVE works unfortunately.

Thanks for the detailed explanation [@attritionorg](#)!

2

attritionorg on Jan 7, 2022

...

MITRE/CVE less so, but other databases look at libraries a bit different. We have to consider "what is the base vuln" vs "how can this be implemented". In some cases we have no idea how a typical integration works which speaks to your comment "99.9x% of usage is not exposed to untrusted users". This is extremely helpful for us since it tell us the likelihood of an attack vector are slim. We'd be more inclined to ignore the report completely unless A) vendor or researcher gives a real-world scenario and we know to be cautious when a researcher provides one B) a CVE is assigned. Then we have to cover it but, we provide value in disclaiming it. In VulnDB, we actually flag entries as "Not a Vuln" in many of these cases and provide a technical note explaining why. That way our users know to ignore the report and move on. Meanwhile, vendors are left to fight with MITRE/CVE to get it disputed which can be an uphill battle.

So again, I really stress that even if it is brief, giving a clear indication of the severity and exploit path, if any, helps us immensely. Thanks!

3

seberg on Jan 7, 2022

Member

...

Thanks! We need to discuss if its worthwhile or not to dispute, I think. Right now I am just a bit curious if something like this would be a good "dispute" (that we could probably reuse verbatim for de-facto all of these):

Not a meaningful vulnerability because triggering the issue seems only plausible if the malicious party already has the privilege to run NumPy commands. Thus, while a bug, it does not present an escalation of privilege.

72 remaining items

Load more



andrewbolster added a commit that references this issue [on Mar 8, 2022](#)

Ignore numpy [CVE-2021-41495](#) until safety db is updated ([#218](#)) ...

Verified

35e8838

seberg on Mar 8, 2022

Member



This was already part of 1.22.2, we just may need to ping the CVE database folks again to update their entry.



seberg removed this from the [1.22.4 release](#) milestone [on Mar 8, 2022](#)



rgommers removed [09 - Backport-Can...](#) [on Mar 9, 2022](#)



rgommers added this to the [1.22.2 release](#) milestone [on Mar 9, 2022](#)

rgommers on Mar 9, 2022

Member



This was already part of 1.22.2, we just may need to ping the CVE database folks again to update their entry.

Yes indeed, thanks - some of the comments above were unclear and it wasn't marked. It's indeed in [1.22.2](#), the backport PR was [gh-20984](#). I have added the milestone now.

One comment above says *"Take into account that the safety check database is updated only once a month. So, you need to wait next month to see something changed there"*. That was a month ago, so I hope it's up-to-date now.



CasperWA added a commit that references this issue [on Mar 25, 2022](#)

Add numpy safety ignore ...

Verified

6932dc4



CasperWA mentioned this [on Mar 25, 2022](#)

[Add numpy safety ignore emmo-repo/domain-cif#96](#)



CasperWA added a commit that references this issue [on Mar 25, 2022](#)

Add numpy safety ignore ([#96](#)) ...

Verified

7a4266c



Zengyf-CVer mentioned this [on May 5, 2022](#)

[Update numpy version to fix CVE vulnerability ultralytics/yolov5#7703](#)



samjzhu added a commit that references this issue [on Jun 27, 2022](#)

Fix for [CVE-2021-41495](#) [numpy/numpy#19038](#)

Verified bbb5adc



CasperWA added 2 commits that reference this issue [on Mar 15, 2023](#)

Remove safety ignore for NumPy ...

Verified d2fd05c

Update to SINTEF/ci-cd v2 ([#105](#)) ...

Verified e6d6a45



jorisvandenbossche mentioned this [on Jun 1, 2023](#)

[Minimum required numpy version \(1.16.6\) has security vulnerability apache/arrow#35846](#)



CasperWA mentioned this [on Oct 17, 2023](#)

[Issue to track the removal of safety ignore 44715 EMMC-ASBL/oteapi-plugin-template#200](#)

[Sign up for free](#) to join this conversation on **GitHub**. Already have an account? [Sign in to comment](#)

Assignees

No one assigned

Labels

00 - Bug

Type

No type

Projects

No projects

Milestone

1.22.2 release

Closed on Feb 12, 2022, 100% complete

Relationships

None yet

Development

Code with agent mode

ENH: review return values for PyArray_DescrNew

[numpy/numpy](#)

Participants

+11

